Anexo de Estruturas de Dados: MGPEB

Missão Aurora Siger | Atividade Integradora, Fase 2 | FIAP

Grupo 3: Jean Melo

Outubro de 2026

Este anexo complementa o relatório técnico e o código mgpeb.py, descrevendo como listas, filas e pilhas foram utilizadas no protótipo do Módulo de Gerenciamento de Pouso e Estabilização de Base (MGPEB), com exemplos concretos extraídos do código.

1. Visão geral

Lista (MODULOS, pousados, em_espera, em_alerta). A lista é a estrutura de cadastro: cada módulo é um dicionário com seus atributos, e a lista MODULOS guarda os seis. Buscas e ordenações recebem uma lista e devolvem uma nova lista, sem alterar a original. As três listas auxiliares recebem os módulos conforme a decisão; por exemplo, após a simulação, pousados contém M02 e M01, e em_alerta contém M04. Como a lista permite acesso por índice, ela é a base da busca binária.

Fila (classe Fila). A fila modela a ordem de atendimento: o primeiro módulo a entrar é o primeiro a ser avaliado (FIFO). A classe tem enfileirar (adiciona no fim), desenfileirar (remove do início), primeiro, vazia e tamanho. Na simulação, a fila final é montada com a lista ordenada por prioridade e combustível (M02, M01, M06, M05, M03, M04) e esvaziada com while not fila.vazia(): m = fila.desenfileirar(). Isso garante que um módulo de prioridade 4 nunca seja avaliado antes de um de prioridade 1, mesmo que tenha chegado à órbita primeiro.

Pilha (classe Pilha). A pilha modela o registro de eventos: o último evento é o primeiro a ser consultado (LIFO). A classe tem empilhar, desempilhar, topo, vazia e tamanho. A cada decisão, eventos.empilhar((m["id"], decisao)) registra o par módulo e decisão. Ao final, eventos.topo() devolve o evento mais recente, ('M04', 'ABORTADO'), e o laço de desempilhar imprime o histórico em ordem inversa. Essa é a ordem natural de uma investigação: começar pela última decisão e voltar no tempo. A mesma pilha permitiria desfazer uma decisão, retirando o evento do topo e devolvendo o módulo à fila.

Estrutura	Papel no MGPEB	Operações usadas	Ordem
Lista	cadastro, resultados de busca e ordenação, listas auxiliares	índice, percorrer, acrescentar	qualquer
Fila	fila de pouso	enfileirar, desenfileirar, primeiro	FIFO
Pilha	registro de eventos para auditoria	empilhar, desempilhar, topo	LIFO

Tabela 4. Resumo das estruturas lineares no projeto.

2. Implementação das estruturas

As classes abaixo são as usadas no protótipo. Ambas encapsulam uma lista Python e expõem apenas as operações que definem cada estrutura, o que impede, por exemplo, que a fila seja acessada fora da ordem FIFO.

```
class Fila:
    """Fila (FIFO): o primeiro modulo a entrar e o primeiro a ser atendido."""

    def __init__(self):
        self.itens = []

    def enfileirar(self, item):
        self.itens.append(item)          # entra no fim

    def desenfileirar(self):
        if self.vazia():
            return None
        return self.itens.pop(0)         # sai do inicio

    def primeiro(self):
        return None if self.vazia() else self.itens[0]

    def vazia(self):
        return len(self.itens) == 0

    def tamanho(self):
        return len(self.itens)

class Pilha:
    """Pilha (LIFO): o ultimo evento registrado e o primeiro a ser consultado."""

    def __init__(self):
        self.itens = []

    def empilhar(self, item):
        self.itens.append(item)          # entra no topo

    def desempilhar(self):
        if self.vazia():
            return None
        return self.itens.pop()          # sai do topo

    def topo(self):
        return None if self.vazia() else self.itens[-1]

    def vazia(self):
        return len(self.itens) == 0
```

```
    def tamanho(self):
        return len(self.itens)
```

3. As estruturas em ação

O trecho abaixo, da função main(), mostra as três estruturas trabalhando juntas: a fila entrega os módulos na ordem de prioridade, as listas auxiliares recebem cada módulo conforme a decisão, e a pilha registra o evento.

```
    # montagem e consumo da fila de pouso (trecho de main)
    fila = Fila()
    for m in fila_final:
        fila.enfileirar(m)
    while not fila.vazia():
        m = fila.desenfileirar()
        decisao, motivos = avaliar_pouso(m, AMBIENTE)
        eventos.empilhar((m["id"], decisao))      # pilha de auditoria
        if decisao == "AUTORIZADO":
            pousados.append(m)                    # lista auxiliar
        elif decisao == "ADIADO":
            em_espera.append(m)
        else:
            em_alerta.append(m)
```

Resultado no cenário simulado: fila esvaziada na ordem M02, M01, M06, M05, M03, M04; listas finais pousados = [M02, M01], em_espera = [M06, M05, M03], em_alerta = [M04]; topo da pilha ('M04', 'ABORTADO'), com os seis eventos impressos do mais recente ao mais antigo.